

优秀学员统计

题目描述

公司某部门软件教导团正在组织新员工每日打卡学习活动，他们开展这项学习活动已经一个月了，所以想统计下这个月优秀的打卡员工。每个员工会对应一个id，每天的打卡记录记录当天打卡员工的id集合，一共30天。

请你实现代码帮助统计出打卡次数top5的员工。加入打卡次数相同，将较早参与打卡的员工排在前面，如果开始参与打卡的时间还是一样，将id较小的员工排在前面。

注：不考虑并列的情况，按规则返回前5名员工的id即可，如果当月打卡的员工少于5个，按规则排序返回所有有打卡记录的员工id。

输入描述

第一行输入为新员工数量N，表示新员工编号id为0到N-1，N的范围为[1,100]

第二行输入为30个整数，表示每天打卡的员工数量，每天至少有1名员工打卡。

之后30行为每天打卡的员工id集合，id不会重复。

输出描述

按顺序输出打卡top5员工的id，用空格隔开。

用例

输入	11 4 4 1 2 2 0 1 7 10 0 1 6 10 10 10
----	---

题目解析

简单排序 题。需要注意的是，排序要素需要记录每个员工第一次打卡日期，作为第二优先级排序。

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const n = parseInt(await readline());
7
8      await readline();
9
10     // key记录员工id, value记录员工打卡天数
11     const freq = {};
12     // key记录员工id, value记录员工第一次打卡是哪天
13     const first = {};
14
15     for (let i = 0; i < 30; i++) {
16         const ids = (await readline()).split(" ");
17
18         for (let id of ids) {
19             if (freq[id] == undefined) {
20                 freq[id] = 0;
21             }
22
23             freq[id]++;
24
25             if (first[id] == undefined) {
26                 first[id] = i;
27             }
28         }
29     }
```

运行

```

30
31 |   const ans = Object.keys(freq)
32 |   .sort((a, b) => {
33 |     // 打卡天数多的员工排在前面
34 |     if (freq[a] !== freq[b]) {
35 |       return freq[b] - freq[a];
36 |     }
37 |
38 |     // 首次打卡早的员工排在前面
39 |     if (first[a] !== first[b]) {
40 |       return first[a] - first[b];
41 |     }
42 |
43 |     // id小的员工排在前面
44 |     return a - b;
45 |   })
46 |   .slice(0, 5) // 取打卡前5名
47 |   .join(" ");
48 |
49 |   console.log(ans);
50 | })();

```

Java算法源码

```

1 | import java.util.HashMap;
2 | import java.util.Scanner;
3 | import java.util.StringJoiner;
4 |
5 | public class Main {
6 |     public static void main(String[] args) {
7 |         Scanner sc = new Scanner(System.in);
8 |
9 |         int n = sc.nextInt();
10 |

```

```

11 int[] count = new int[30];12 |         for (int i = 0; i < 30; i++) {
13     count[i] = sc.nextInt();
14 }
15
16 // key记录员工id, value记录员工打卡天数
17 HashMap<Integer, Integer> freq = new HashMap<>();
18 // key记录员工id, value记录员工第一次打卡是哪天
19 HashMap<Integer, Integer> first = new HashMap<>();
20
21 for (int i = 0; i < 30; i++) {
22     for (int j = 0; j < count[i]; j++) {
23         int id = sc.nextInt();
24
25         freq.put(id, freq.getOrDefault(id, 0) + 1);
26         first.putIfAbsent(id, i);
27     }
28 }
29
30 StringJoiner sj = new StringJoiner(" ");
31
32 freq.keySet().stream().sorted((idA, idB) -> {
33     int freqA = freq.get(idA);
34     int freqB = freq.get(idB);
35
36     // 打卡天数多的员工排在前面
37     if (freqA != freqB) {
38         return freqB - freqA;
39     }
40
41     int firstA = first.get(idA);
42     int firstB = first.get(idB);
43
44     if (firstA != firstB) {
45         // 首次打卡早的员工排在前面
46         return firstA - firstB;
47     } else {
48         // id小的员工排在前面

```

```

49         return idA - idB;
50     }
51     }).limit(5).forEach(id -> sj.add(id + "")); // 取打卡前5名
52
53     System.out.println(sj);
54 }
55 }

```

Python算法源码

```

1  if __name__ == '__main__':
2      input()
3      input()
4
5      # key记录员工id, value记录员工打卡天数
6      freq = {}
7      # key记录员工id, value记录员工第一次打卡是哪天
8      first = {}
9
10     for day in range(30):
11         ids = list(map(int, input().split()))
12
13         for id in ids:
14             freq.setdefault(id, 0)
15             freq[id] += 1
16
17             first.setdefault(id, day)
18
19     # 打卡天数多的员工排在前面, 若打卡天数新题, 则首次打卡早的员工排在前面, 若首次打卡日期相同, 则id小的员工排在前面
20     # 取前5名
21     print(" ".join(map(str, sorted(freq.keys(), key=lambda id: (-freq[id], first[id], id))[:5])))

```

C算法源码


```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  #define MAX_N 100
6
7  // 数组索引对应员工id, 数组元素对应员工打卡天数
8  int freq[MAX_N];
9  // 数组索引对应员工id, 数组元素对应员工首次打卡日期 (哪一天)
10 int first[MAX_N];
11
12 int cmp(const void *a, const void *b) {
13     int A = *(int *) a;
14     int B = *(int *) b;
15
16     // 打卡天数多的员工排在前面
17     if (freq[A] != freq[B]) {
18         return freq[B] - freq[A];
19     }
20
21     // 首次打卡早的员工排在前面
22     if (first[A] != first[B]) {
23         return first[A] - first[B];
24     }
25
26     // id小的员工排在前面
27     return A - B;
28 }
29
30 int main() {
31     int n;
32     scanf("%d", &n);
33
34     int count[30] = {0};
35     for (int day = 0; day < 30; day++) {
36         scanf("%d", &count[day]);
```

```
37     }38 |
39     for (int id = 0; id < n; id++) {
40         freq[id] = 0;
41         first[id] = -1;
42     }
43
44     for (int day = 0; day < 30; day++) {
45         for (int i = 0; i < count[day]; i++) {
46             int id;
47             scanf("%d", &id);
48
49             freq[id]++;
50
51             if (first[id] == -1) {
52                 first[id] = day;
53             }
54         }
55     }
56
57     // 收集有打卡的员工id
58     int ids[n];
59     int ids_size = 0;
60
61     for (int id = 0; id < n; id++) {
62         if (freq[id] > 0) {
63             ids[ids_size++] = id;
64         }
65     }
66
67     qsort(ids, ids_size, sizeof(ids[0]), cmp);
68
69     // 取打卡前5名
70     for (int i = 0; i < fmin(ids_size, 5); i++) {
71         printf("%d ", ids[i]);
72     }
73
74     return 0;
```

```
75 | }  
76 |
```

C++ 算法源码

```
1  #include <bits/stdc++.h>  
2  
3  using namespace std;  
4  
5  int main() {  
6      int n;  
7      cin >> n;  
8  
9      vector<int> count(30);  
10     for (int day = 0; day < 30; day++) {  
11         cin >> count[day];  
12     }  
13  
14     // key记录员工id, value记录员工打卡天数  
15     map<int, int> freq;  
16     // key记录员工id, value记录员工第一次打卡是哪天  
17     map<int, int> first;  
18  
19     for (int day = 0; day < 30; day++) {  
20         for (int i = 0; i < count[day]; i++) {  
21             int id;  
22             cin >> id;  
23  
24             freq[id]++;  
25  
26             if (first.find(id) == first.end()) {  
27                 first[id] = day;  
28             }  
29         }  
30     }
```

```
31 |
32 |     vector<int> ids;
33 |     for (const auto &p: freq) {
34 |         ids.emplace_back(p.first);
35 |     }
36 |
37 |     sort(ids.begin(), ids.end(), [&](int a, int b) {
38 |         // 打卡天数多的员工排在前面
39 |         if (freq[a] != freq[b]) {
40 |             return freq[a] > freq[b];
41 |         }
42 |
43 |         // 首次打卡早的员工排在前面
44 |         if (first[a] != first[b]) {
45 |             return first[a] < first[b];
46 |         }
47 |
48 |         // id小的员工排在前面
49 |         return a < b;
50 |     });
51 |
52 |     // 取打卡前5名
53 |     for (int i = 0; i < min((int) ids.size(), 5); i++) {
54 |         cout << ids[i] << " ";
55 |     }
56 |
57 |     return 0;
58 | }
```